# Uzair Nawaz

512-244-3928 | uzair.nawaz@utexas.edu | github.com/uzairnawaz

## Education

**University of Texas at Austin** — Austin, TX
*B.S. Computer Science Honors (Turing Scholar), GPA: 3.89/4.00* — *Graduating Spring 2027*
Courses: Virtualization, Neural Networks, Hon Quantum Computing, Hon Algorithms, Programming for Performance, Hon Computer Security, Hon Operating Systems, Hon Computer Architecture, Data Structures, Lin Algebra

## Technical Skills

**Languages**: C/C++, Python, Java, OCaml, Rust, x86/ARM Assembly, Bash, JavaScript, HTML/CSS, SQL, Verilog
**Tools**: Git, CMake, Makefile, GDB, JIRA, Confluence, Unity Game Engine, Arduino Microcontroller

## Experience

**Jane Street** — May 2026 – Aug 2026
*Software Engineer Intern* — *New York City, NY*

- Redesigned software deployment system for trading servers, replacing an hourly polling job with a long-running service written in **OCaml** that continuously monitors fleet state and makes rollout decisions in real time.
- Improved rollout scheduling by using server uptime/schedule data to trigger roll evaluations at optimal times instead of a fixed hourly cadence, increasing responsiveness of the deployment pipeline.
- Decoupled deployment source-of-truth from the app's repo, eliminating rebuilds on every version change and reducing deployment time by 75%.

**The University of Texas at Austin** — Spring 2025 & Fall 2025
*Undergraduate Teaching Assistant (CS 439: Operating Systems)* — *Austin, TX*

- Planned, graded, and answered questions related to programming assignments and quizzes for about 65 students.

**Google** — May 2025 – Aug 2025
*Associate Software Developer Intern - Compilers, Runtimes, and Toolchains Team* — *Sunnyvale, CA*

- Contributed to LLVM-libc, an open-source standard library for C which is part of The LLVM Project.
- Designed, implemented, and tested a system to perform conversion between character encodings in **C++**.
- Contributed to the pthreads implementation, designing a reusable barrier for thread synchronization.
- 2025 LLVM Conference: Presented "Project Widen Your Char-izons: Adding Wchar Support to LLVM-libc."

**Keysight Technologies** — May 2024 – Aug 2024
*Network Visibility Software Intern* — *Austin, TX*

- Developed a staged upgrade process for loading new software on network switches. Allowed network switches to remain online for a large part of the software upgrade process, decreasing downtime by about 40%.
- Reorganized one-shot **Bash** upgrade scripts into logical stages that could be run independently.
- Improved existing web GUI using ExtJS to display software installation status and future actions.
- Implemented new features to **Java** REST API, allowing users to trigger specific stages of the installation process.

## Projects

**LDOS ML Compiler Research Project** | *Python, PyTorch, Triton, C, Linux Kernel*

- Developed a process to compile PyTorch models using Triton and perform inference on them within the kernel.
- Increased the feasability of using ML-based OS policies instead of the existing heuristic approaches.

**Chess Engine** | *C++, Google Test*

- Wrote a UCI-compatible chess engine that applies heuristics and alpha-beta pruning to choose the best move.
- Represented the board as a collection of bitboards (64-bit integers with each bit representing a square), computing moves through efficient bitwise operations. Utilized Google Test framework to verify move generation.

**Object-Oriented FUN Compiler** | *C++, ARM Assembly, Makefile*

- Built compiler to translate a toy language to ARM assembly. Supports conditionals, loops, functions, and classes.
- Implemented compile-time optimizations such as constant folding and tail call elimination by constructing an AST.

**Pipelined Processer** | *SystemVerilog*

- Designed a 6-staged pipelined processor with an ISA that included branches, memory read/writes (including misaligned addresses), and arithmetic. Implemented a branch predictor to reduce pipeline flushes.

**Network Card Driver** | *C++*

- Implemented a driver for the E1000 family of network cards. Supported both transmitting and receiving.